



Lab 9: Pointers

CSE 4108

Structured Programming I Lab



Lab Tasks

1. Pointers:

Write a program that declares and initializes (to any value you like) a double, an int, and a char. Next declare and initialize a pointer to each of the three variables. Your program should then print the address of, and value stored in, and the memory size (in bytes) of each of the variables and their corresponding pointers. Here's an example of the desired output:

```
The address of char c is 0x61feff
The address of int i is 0x61fef4
The address of double d is 0x61fee8
The address of char* pc is 0x61fef8
The address of int* pi is 0x61fef0
The address of double* pd is 0x61fee4
```

```
The value of char c is a
The value of int i is 1
The value of double d is 2.500000
The value of char* pc is 6422271
The value of int* pi is 6422260
The value of double* pd is 6422248
```

```
The size of char is 1
The size of int is 4
The size of double is 8
The size of char* is 4
The size of int* is 4
The size of double* is 4
```

Note: Use the "0x%x" format specifier to print the addresses in hexadecimal.

[Hint] Use the function sizeof() to get the memory size.

2. Swap:

Write a program that contains the following function:

```
void swap (int *x, int *y);
```

The function will swap the values pointed by x and y. The main function will pass addresses of two variables which need to be swapped. And again, remember, a function does what it says. So the function itself should only do the swapping, it SHOULD NOT scan or print anything.

3. I Reminder You:

In C, you have the % or the remainder operator to find remainders. However, it only works with integral values. What if we want to find out the quotient and remainder of a float division? For instance, if we divide 4.6 by 1.5, we can see that the quotient is 3 and remainder is 0.1.

Write a function in C that would take the dividend and divisor of a float division as input and give back the quotient and remainder. Since we need two things back, we need to use pointers. Also, remember, the sign of the remainder depends on the sign of the dividend. If the dividend is non-negative, remainder will be non-negative. If the dividend is negative, remainder will be negative. Remember to test with all 4 types of combinations of input with non-negative and negative dividends and divisors. The function declaration looks like below:

```
void fdiv(double dividend, double divisor, double *quotient,  
         double *remainder);
```

Sample Input

Enter dividend: -5.2

Enter divisor: 2.5

Sample Output

Quotient: -2

Remainder: -0.2

NOTE: You are not allowed to use the fmod function for this task.

[Hint] You can use floor, ceil etc. functions from math library to help you find the quotient first. Then you can use that whole value quotient to find the remainder.

4. **Back to Texas:**

Convenience Delivery Shop (CDS) of Imperial University of Texas (IUT) is a small retail business that stocks a range of everyday items such as snack foods, confectionery, soft drinks, toiletries, etc. Everyday numerous students get their daily goods from CDS. After purchasing an item, whenever a student gives a note of large denomination (like \$500 for example), the shop owner Hashin bhai faces a huge trouble. He can easily calculate the amount to be returned as (totalCost-totalPaid). But he does not know how to pay that amount using smallest number of denominations.

Hashin bhai knows you're a great programmer and asked for your help. You need to write a function that takes totalPaid and totalCost as input (in U.S. dollar) and then gives back the smallest number of \$20, \$10, \$5, and \$1 bills required to be returned. Hashin bhai has ensured you that he will not enter any fractional value as input.

The function gives back the amount of different bills. Since there are 4 different types of bill amounts to give back, pointers are required. The declaration of the function is like below:

```
void getBillAmounts(int totalPaid, int totalCost, int *twenties,  
                   int *tens, int *fives, int *ones);
```

Sample Input

Enter total paid: 500

Enter total cost: 407

Sample Output

\$20 bills: 4

\$10 bills: 1

\$5 bills: 0

\$1 bills: 3

5. Mid Term:

Write a function in C that would take an array which is already sorted in ascending order, the array length and a pointer as input and return a pointer to the median of the value(s).

Already sorted in ascending order means you do not need to worry about sorting the array yourself. The values will be sorted from small to large when sent to the function.

Now even length arrays basically have two medians, the middle two elements. Generally their average is taken. But since we are returning pointer to the median, we need to inform one more thing. Whether there is a second median or not, in other words, whether the array is of even length or not. If there is a second median, we can add 1 with the returned pointer (which points to the first median) to get the second median pointer. So, the function will additionally take an integer (Boolean) pointer as input and return 1 (true) if there is a second median or 0 (false) if there is not. The declaration of the function looks like below.

```
double* getPtrToMedian(double arr[], int len, int  
                        *secondMedian);
```

To clarify, if the function is called on, {1 3 4 5 6 7}, it will return pointer to the 3rd element with value 4 and set *secondMedian to 1. But if the function is called on {20 40 50}, it will return pointer to the 2nd element 40 and set *secondMedian to 0.

Design the input and output accordingly to test your program. Again, do the scanning and printing in main function.